Linear Discriminant Analysis

Nathan Seto

Packages

```
library(recipes)
library(readr)
library(dplyr)
library(caret)
library(ggplot2)
library(parsnip)
library(purrr)
library(discrim)
library(tidyr)
library(lda)
library(VIM)
library(MASS)
library(pander)
```

Loading Data

```
train = read_csv("train.csv")

# LDA with removal of NA in response
train_clean = train[!is.na(train$sii), ]

colnames(train) = gsub("-", "_", colnames(train))
train_clean = train_clean[, !grepl("Season", names(train))]
train_clean$sii = factor(train_clean$sii) # Making sure `sii` is a factor with 4 levels (plus extra lev

# Imputation
categorical_vars = train_clean[, sapply(train_clean, function(x) is.factor(x) | is.character(x))]
pciat_columns = grep("PCIAT", names(train_clean), value = TRUE)
pciat_columns = pciat_columns[pciat_columns != "PCIAT_PCIAT_Total"]
categorical_vars = cbind(categorical_vars, train_clean[, pciat_columns])

categorical_vars = categorical_vars %>%
  mutate(across(everything(), ~ replace(., is.na(.), "Missing")))

quantitative_vars = train_clean[, sapply(train_clean, is.numeric)]

quantitative_vars_imputed = quantitative_vars %>%
  mutate(across(where(is.numeric), ~ ifelse(is.na(.), median(., na.rm = TRUE), .)))

# Accounting for collinearity
correlation_matrix = cor(quantitative_vars_imputed)
```

```r
  high_corr_pairs = findCorrelation(correlation_matrix, cutoff = 0.8, verbose = TRUE)
```

```
## Compare row 66  and column  48 with corr  0.822
##   Means:  0.287 vs 0.141 so flagging column 66
## Compare row 5  and column  6 with corr  0.809
##   Means:  0.217 vs 0.137 so flagging column 5
## Compare row 6  and column  4 with corr  0.845
##   Means:  0.202 vs 0.135 so flagging column 6
## Compare row 63  and column  61 with corr  0.838
##   Means:  0.2 vs 0.133 so flagging column 63
## Compare row 41  and column  42 with corr  0.994
##   Means:  0.195 vs 0.131 so flagging column 41
## Compare row 42  and column  39 with corr  0.997
##   Means:  0.174 vs 0.129 so flagging column 42
## Compare row 39  and column  32 with corr  0.993
##   Means:  0.16 vs 0.128 so flagging column 39
## Compare row 32  and column  43 with corr  0.994
##   Means:  0.147 vs 0.127 so flagging column 32
## Compare row 43  and column  31 with corr  1
##   Means:  0.128 vs 0.127 so flagging column 43
## Compare row 31  and column  34 with corr  1
##   Means:  0.11 vs 0.127 so flagging column 34
## Compare row 31  and column  33 with corr  0.999
##   Means:  0.095 vs 0.128 so flagging column 33
## Compare row 31  and column  37 with corr  0.979
##   Means:  0.079 vs 0.129 so flagging column 37
## Compare row 31  and column  40 with corr  0.999
##   Means:  0.063 vs 0.131 so flagging column 40
## Compare row 31  and column  29 with corr  0.99
##   Means:  0.046 vs 0.134 so flagging column 29
## Compare row 18  and column  16 with corr  0.867
##   Means:  0.127 vs 0.137 so flagging column 16
## Compare row 22  and column  24 with corr  0.914
##   Means:  0.106 vs 0.137 so flagging column 24
## Compare row 68  and column  67 with corr  0.996
##   Means:  0.094 vs 0.139 so flagging column 67
## All correlations <= 0.8
```

```r
    quantitative_vars_filtered = quantitative_vars_imputed[, -high_corr_pairs]

train_data = bind_cols(categorical_vars, quantitative_vars_filtered)

quantitative_vars_filtered = quantitative_vars_filtered %>% mutate(sii = train_data$sii)

# Split data into train and test
set.seed(445)
trainIndex = createDataPartition(train_data$sii, p = 0.8, list = FALSE)
train_set = train_data[trainIndex, ]
test_set = train_data[-trainIndex, ]
```

LDA Model

```r
lda_data_train = train_set
  lda_data_train$sii = as.factor(lda_data_train$sii)
```

```r
    lda_data_train = subset(lda_data_train, select = -id)

lda_model_train = lda(sii ~ ., data = lda_data_train)

lda_predictions_train = predict(lda_model_train, lda_data_train)
  lda_data_train$predicted_sii = lda_predictions_train$class

confusion_matrix_train = table(lda_data_train$sii, lda_data_train$predicted_sii)
print("Train Confusion Matrix:")
```

```
## [1] "Train Confusion Matrix:"
```

```r
print(pander(confusion_matrix_train))
```

```
## [1] "\n--------------------------------\n      0      1     2    3  \n-------- ------ ----- ----
## attr(,"class")
## [1] "knit_asis"
## attr(,"knit_cacheable")
## [1] NA
```

```r
accuracy_train = sum(diag(confusion_matrix_train)) / sum(confusion_matrix_train)
print(paste("Train Accuracy:", round(accuracy_train * 100, 2), "%"))
```

```
## [1] "Train Accuracy: 98.81 %"
```

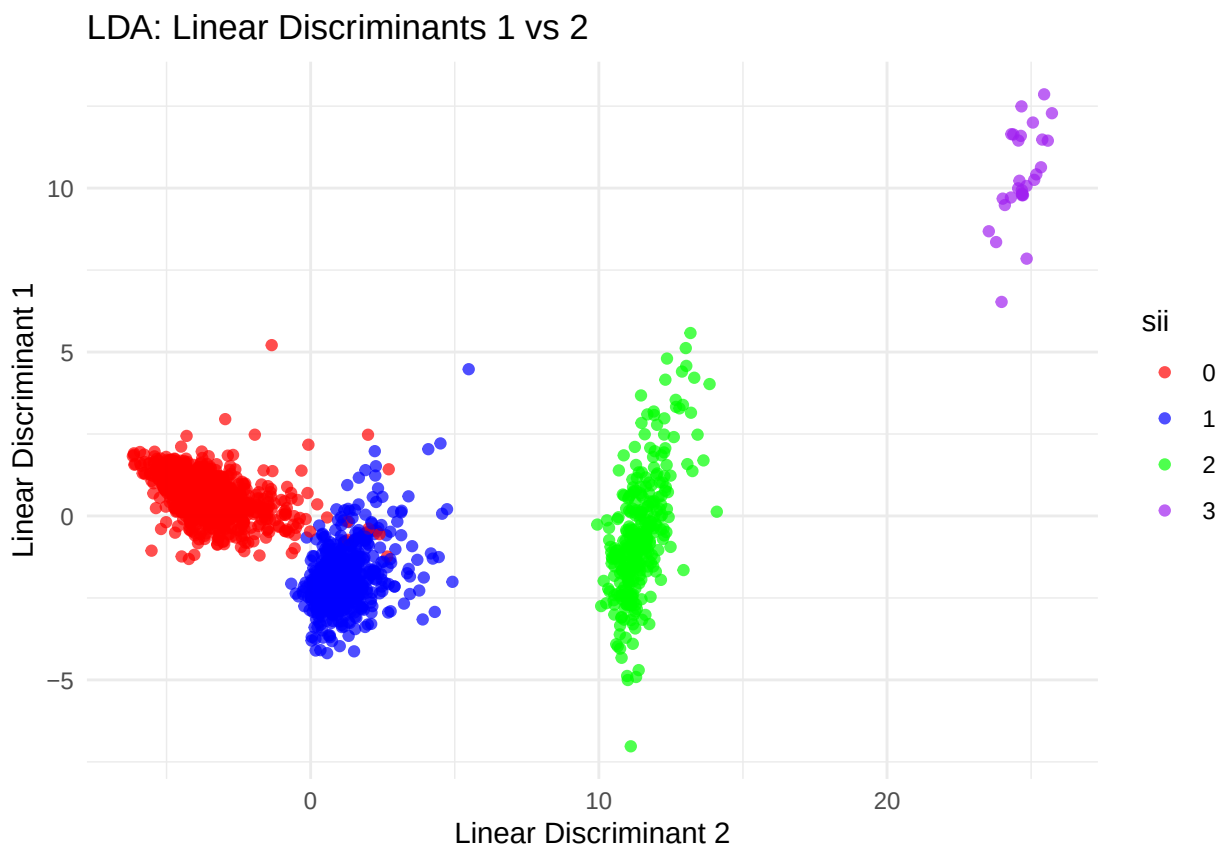
```r
error_rate_train = 1 - accuracy_train
print(paste("Train Error Rate:", round(error_rate_train * 100, 2), "%"))
```

```
## [1] "Train Error Rate: 1.19 %"
```

Visualization

```r
lda_discriminants = lda_predictions_train$x
lda_discriminants = as.data.frame(lda_discriminants)

if (ncol(lda_discriminants) >= 2) {
  lda_data_train$LD1 = lda_discriminants[, 1]
  lda_data_train$LD2 = lda_discriminants[, 2]
  plot = ggplot(lda_data_train, aes(x = LD1, y = LD2, color = sii)) +
    geom_point(alpha = 0.7) +
    labs(title = "LDA: Linear Discriminants 1 vs 2",
         x = "Linear Discriminant 2",
         y = "Linear Discriminant 1") +
    theme_minimal() +
    scale_color_manual(values = c("red", "blue", "green", "purple"))
} else {
  print("Insufficient linear discriminants for visualization.")
}
print(plot)
```



Predicting on Test Set

```
lda_data_test = test_set
  lda_data_test$sii = as.factor(lda_data_test$sii)
    lda_data_test = subset(lda_data_test, select = -id)

lda_model_test = lda(sii ~ ., data = lda_data_test)

# Make prediction
lda_predictions_test = predict(lda_model_test, lda_data_test)
  lda_data_test$predicted_sii = lda_predictions_test$class

# Confusion Matrix
confusion_matrix_test = table(lda_data_test$sii, lda_data_test$predicted_sii)
print("Test Confusion Matrix:")
```

```
## [1] "Test Confusion Matrix:"
```

```
pander(confusion_matrix_test)
```

	0	1	2	3
0	314	4	0	0
1	0	146	0	0
2	0	0	75	0
3	0	0	0	6

```r
accuracy_test = sum(diag(confusion_matrix_test)) / sum(confusion_matrix_test)
print(paste("Test Accuracy:", round(accuracy_test * 100, 2), "%"))
```

```
## [1] "Test Accuracy: 99.27 %"
```

```r
error_rate_test = 1 - accuracy_test
print(paste("Test Error Rate:", round(error_rate_test * 100, 2), "%"))
```

```
## [1] "Test Error Rate: 0.73 %"
```